

LAYER 10 OF 13

CACHING CDN

Making Your App Load Fast Everywhere

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 10: Caching & CDN—the techniques that make your app feel instant instead of sluggish. Caching means saving a copy of something so your app doesn't have to fetch it again. The first time a user loads your homepage, your app gathers all the data, builds the page, and sends it. Caching saves that result so the second time anyone asks for the same page, it's already ready—no rebuilding required. It's the difference between cooking a meal from scratch every time and having leftovers ready to microwave.

A CDN—Content Delivery Network—takes caching a step further by putting copies of your app's files on servers all around the world. Without a CDN, every user's request travels to your one server, wherever it lives. A user in Tokyo requesting data from a server in Virginia waits for that round trip across the Pacific Ocean. With a CDN, copies of your files sit on servers in Tokyo, London, Sydney, and dozens of other cities. Users load from the server closest to them, and the result is dramatic: pages that took two seconds now load in a fraction of that.

This kit walks you through three levels of caching mastery. Tier 1 teaches you the basics—browser caching, CDN setup, and how to tell your AI coding tool to make your app fast. Tier 2 gets into server-side caching, cache invalidation (the tricky part—knowing when to throw away old copies), and performance monitoring. Tier 3 covers enterprise-scale strategies like edge computing, multi-region caching, and cache architecture for high-traffic platforms.

Why It Matters

Speed isn't a nice-to-have. Studies consistently show that users start abandoning websites after just a few seconds of loading time. If your app takes three seconds to load a page, a significant percentage of users will leave before they ever see it. And it compounds: slow apps get fewer users, fewer users mean less feedback, less feedback means slower improvement. Speed is a competitive advantage that affects everything downstream.

Here's why this matters specifically for vibecoders: AI coding tools build apps that work, but they rarely optimize for speed automatically. AI will load a full-size image when a thumbnail would do. It will fetch data from the database on every page load instead of caching the result. It will serve your app from one server when a CDN could serve it globally. These aren't bugs—the app works fine. But it works slowly. This kit teaches you what to ask for so your AI builds an app that doesn't just work, but loads fast for everyone, everywhere.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier

Who It's For

What You Prove

Tier 1

Solo builders shipping their first app to real users

You can set up basic caching, deploy through a CDN, optimize images and static files, and verify that your app loads quickly for users in different locations.

Tier 2

Growing apps with dynamic content and increasing traffic

You can implement server-side caching, manage cache invalidation, monitor cache performance, and balance freshness with speed across your application.

Tier 3

High-traffic platforms with global users and strict performance requirements

You can architect caching at scale—edge computing, multi-layer cache strategies, cache consistency across regions, and automated performance optimization.

TIER 1 CERTIFICATION GOAL

You can set up browser caching, deploy your app through a CDN, optimize images and static files, and verify that your app loads quickly—all by directing your AI coding tool with the right prompts.

What You Need to Know

You don't need to understand the internals of caching systems. You need to know what caching is, where it helps, and what to ask your AI tool to set up.

Browser caching (your user's device does the work): When someone visits your app, their browser downloads files—images, fonts, JavaScript, CSS. Browser caching tells the browser "save these files locally so you don't have to download them again next time." This is controlled by cache headers—instructions your server sends that say "keep this file for 7 days" or "always check for a new version." Your hosting platform (Vercel, Netlify, Cloudflare Pages) usually sets these automatically, but you should verify they're configured correctly.

CDN basics (copies of your app, everywhere): A CDN is a network of servers around the world that store copies of your app's static files—images, fonts, CSS, JavaScript. When a user in Brazil loads your app, they get these files from a server in South America instead of waiting for them to cross the ocean from your main server. Most modern hosting platforms include a CDN automatically, but you need to confirm it's active and serving your files.

Static vs. dynamic content: Static content is stuff that doesn't change per user—your logo, your fonts, your styling files. This is perfect for caching. Dynamic content changes depending on who's viewing it—your dashboard, your profile, your shopping cart. This needs more careful caching because you can't show User A a cached version of User B's dashboard. Knowing the difference helps you tell AI what to cache and what not to.

Image optimization (the biggest speed win): Images are usually the heaviest files your app loads. A single unoptimized photo can be 5MB—larger than all your other files combined. Image optimization means serving the right size (don't load a 4000px image for a 400px thumbnail), the right format (WebP is smaller than JPEG for the same quality), and with lazy loading (images below the fold don't load until the user scrolls to them). Ask your AI tool to implement all three.

How to measure speed: You can't improve what you don't measure. Lighthouse (built into Chrome DevTools) gives your app a performance score and specific recommendations. The key number is Largest Contentful Paint (LCP)—how long until the biggest visible element loads. Under 2.5 seconds is good. Over 4 seconds is poor. Run Lighthouse on your app and know your numbers.

Your Toolkit

Your AI coding tool builds the caching logic. These tools and platforms help you set it up and verify it works.

Vercel, Netlify, or Cloudflare Pages: Modern hosting platforms that include CDN, automatic cache headers, and image optimization built in. These handle most Tier 1 caching needs out of the box.

Chrome DevTools (Lighthouse and Network tab): Lighthouse gives you a performance score and recommendations. The Network tab shows you exactly what files are loading, how big they are, and whether they're being cached or re-downloaded.

Next.js Image component or similar: Built-in image optimization that automatically resizes, reformats, and lazy-loads images. Your AI tool should use this instead of plain image tags.

Certification Exam Topics

Every exam question is scenario-based. You'll see a real situation and need to identify what's right, what's wrong, or what to do next.

Slow first load: Your app takes 4 seconds to load on the first visit. Lighthouse shows images are the biggest problem. What three things do you ask your AI tool to do?

Cache verification: How do you check whether your app's static files are actually being cached by the browser instead of re-downloaded on every visit?

CDN check: Your app is hosted on Vercel. A user in Australia reports slow loading times. How do you verify whether the CDN is serving files from a nearby server?

Static vs. dynamic: Your app has a product catalog page and a user dashboard. Which one is a good candidate for aggressive caching, and why?

Image formats: Your app loads 20 product images as full-size PNGs. What specific changes should you ask AI to make, and what performance improvement should you expect?

Lighthouse scores: You run Lighthouse and get a performance score of 45. What are the first two metrics you should look at, and what do they tell you?

Repeat visits: A user visits your app for the second time and it loads noticeably faster than the first time. What's happening, and why is this a good sign?

Cache headers: Your AI tool deployed your app but didn't set cache headers on your CSS and JavaScript files. What's the impact, and what should those headers say?

Common Pitfalls

These are the caching mistakes vibecoders make most often. They're easy to miss because the app still works—it's just slower than it should be. If you recognize any of these in your own app, fix them before sitting for the exam.

Serving full-size images when thumbnails would do. A single unoptimized image can be larger than your entire app's code. Ask your AI tool to use responsive images, WebP format, and lazy loading on every image.

Not checking whether your CDN is actually active. You deployed to a platform that offers a CDN, but did you verify it's serving files from edge locations? Check your response headers to confirm files come from the CDN, not just your origin server.

Caching dynamic data without an invalidation strategy. Caching your database queries makes things fast—until someone updates their data and the app shows stale information for hours. Every cached item needs a plan for when and how it gets refreshed.

Never measuring your app's actual load time. You tested it on your fast laptop, on your home internet, sitting ten miles from the server. Real users are on slower connections, older phones, and different continents. Use Lighthouse and real-user monitoring to get honest numbers.

Deploying without cache-busting for CSS and JavaScript. You push an update, but users still see the old version because their browser cached the old files. Cache-busting (adding version numbers to filenames) ensures users always get the latest version after a deploy.

Treating caching as an afterthought instead of a design decision. Caching works best when planned from the start. Retrofitting caching onto a slow app is harder and less effective than building with caching in mind from day one.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Have you run Lighthouse on your deployed app and know your performance score and LCP time?

Are your images optimized—right format (WebP), right size (not larger than displayed), and lazy loaded?

Have you verified that your CDN is active and serving static files from edge locations?

Do your static assets (CSS, JS, fonts) have cache headers set so browsers don't re-download them on every visit?

Can you explain the difference between static content (cacheable for everyone) and dynamic content (varies per user)?

Have you tested your app's load time from a location far from your server, not just from your own device?

Do you know what cache-busting is and can you verify that updated files get served instead of stale cached versions?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick caching and performance health check on your app. It checks the same things the certification exam covers.

Review my app's caching and performance setup. For each item, tell me pass or fail with a specific example:

Image optimization: Are all images served in modern formats (WebP/AVIF), properly sized for their display dimensions, and lazy loaded below the fold?

CDN configuration: Are static assets being served from a CDN with appropriate cache headers? Check response headers for CDN indicators.

Cache headers: Do static files (CSS, JS, fonts, images) have long cache lifetimes with cache-busting on deploy?

Dynamic content caching: Are frequently-accessed database queries or API responses being cached, with a clear invalidation strategy?

Performance metrics: What is the current Lighthouse performance score and Largest Contentful Paint (LCP) time?

Font loading: Are web fonts loaded efficiently (preloaded, display:swap) so they don't block page rendering?

Code splitting: Is the app loading only the code needed for the current page, or downloading everything upfront?

Give me an overall score out of 7 and list the top 3 performance improvements to make first.

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 10 certification exam at your target tier.

The best way to prepare: run Lighthouse on your deployed app right now and look at the score. If it's below 90, there's work to do. Check your images, verify your CDN is active, and ask your AI tool to optimize your biggest performance bottlenecks. Speed improvements are immediately visible and measurable—making them some of the most satisfying proof of work in the entire certification.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai

© 2026 Matt Murphy .AI. All rights reserved.